forge-harness: Engineering Methods for Robust AI Collaboration Harnesses

권성진 (Kwon Sungjin)
Independent Researcher
akaa1941@gmail.com

Abstract

AI collaboration harnesses — structured environments of prompts, skills, agents, and memory files that orchestrate AI-assisted work — have become primary determinants of AI output quality. Empirical analysis finds that 98.4% of Claude Code session content is harness infrastructure rather than direct task instructions [VILA-Lab, 2026]. Despite this, harness engineering lacks principled validation and evolution methods: harnesses are typically built and evaluated by the same practitioner, documentation drifts from implementation, phantom claims accumulate in AI-generated content, and session learnings are lost between working sessions.

We present **forge-harness**, an open harness engineering toolkit that addresses these gaps through four complementary methods: (1) **steel-quench**, a multi-wave adversarial validation framework that separates attack from defense to systematically surface structural defects; (2) **source-grounding-audit**, a phantom claim detection method that demands file-path, commit-hash, or measured-value evidence for every factual claim in harness content; (3) **harvest-loop**, a session-to-harness self-evolution pipeline that transforms session learnings into validated skill upgrades without accumulating technical debt; and (4) **sim-conductor**, a pre-deployment simulation framework that validates harness transferability before non-owner adoption.

Applied to forge-harness itself — a self-verification scenario — the four methods resolved 10 S/A-grade structural defects, detected 3 phantom capability claims, absorbed 5 external session contributions as validated skill upgrades, and confirmed multi-user autonomous operation across three independent deployments. The methodology independently converged on the same outer-loop architecture as harness-evolver [Christi, 2026] and the Stanford Meta-Harness [arXiv:2603.28052], validating the structural approach across independent implementations. forge-harness is available at https://github.com/chrono-code/forge-harness.

---

1. Introduction

AI collaboration harnesses have evolved from simple prompt templates into layered systems comprising memory files, skill libraries, agent dispatch protocols, domain knowledge bases, and behavioral rules. As these harnesses grow in complexity, four specific failure modes recur across practitioner deployments:

F1 — Self-referential validation: The practitioner who builds the harness also evaluates it. A harness that passes its own internal checks may harbor defects visible only to an adversarial evaluator. This is structurally analogous to the penetration-testing problem in software security.

F2 — Phantom claim accumulation: AI-generated harness content (skill documentation, capability descriptions, validation evidence) frequently references files, measurements, or capabilities that do not exist. These "phantom claims" pass informal review and accumulate silently, eroding the reliability of harness content over time.

F3 — Session learning loss: Valuable patterns discovered during working sessions — failure modes caught, successful approaches confirmed, defects identified — are rarely absorbed into the harness systematically. They exist in session transcripts and are lost when context resets.

F4 — Unvalidated transfer: A harness developed by one practitioner may exhibit silent failures when adopted by others — different vocabulary triggering skills, different mental models of skill boundaries, different baseline contexts. Pre-deployment validation of transferability is typically absent.

We introduce **forge-harness**, a toolkit addressing each failure mode through a dedicated method. The paper is structured as follows: Section 2 reviews related work, Section 3 describes the four methods, Section 4 presents evaluation results, Section 5 discusses limitations and positioning, and Section 6 concludes.

Our contributions are: - Four complementary harness engineering methods addressing F1–F4 (Section 3) - The **Sandboxed Adversary principle**, explaining structural convergence of adversarial validation (Section 3.1) - The **phantom claim taxonomy**, classifying three categories of AI-generated false claims in harness content (Section 3.2) - The **synthesizer gate**, a cross-validation step preventing both over-addition and under-addition in harness evolution (Section 3.3) - Empirical application to forge-harness itself, with independent architectural convergence evidence (Section 4)

---

2. Background and Related Work

2.1 The Harness Infrastructure Problem

Chen et al. [2026] analyzed 10,000 Claude Code sessions and found that 98.4% of token content constitutes harness infrastructure — CLAUDE.md rules, memory files, skill invocations, context bridges — rather than direct task instructions [arXiv:2604.14228]. Several concurrent works confirm this framing. "Code as Agent Harness" [arXiv:2605.18747] presents a 43-author study positioning harness code as the primary determinant of agent performance. Sylph.AI's "The Last Harness You'll Ever Build" [arXiv:2604.21003] introduces a self-evolving harness architecture targeting persistent cross-session knowledge. The Stanford IRIS Lab Meta-Harness [arXiv:2603.28052] demonstrates an outer-loop system achieving +7.7 benchmark points at 4× fewer tokens through automated harness code optimization.

Recent empirical work extends this foundation. Ning et al. [arXiv:2605.25665] present a two-pass meta-engineering framework for harness validation with four-way verdict arbitration (TP/FP/UNKNOWN/CONFLICT). Liu et al. [arXiv:2605.26302] identify four mechanisms of harness knowledge aging: compression aging (context window limits), interference aging (conflicting updates), revision aging (documentation drift), and maintenance aging (orphaned references). Min et al. [arXiv:2605.27575] introduce signal-driven harness adaptation for infrastructure-as-code environments. Park et al. [arXiv:2605.28065] propose SpatialBench for joint model-harness pair evaluation, arguing that harness quality dominates model selection in determining agent performance.

2.2 Adversarial Testing and Hallucination Detection

Red-teaming [Perez et al., 2022] and constitutional AI [Bai et al., 2022] apply adversarial pressure to AI model outputs. Our work applies analogous pressure one layer up: to the harness infrastructure itself, not to model outputs. Hallucination detection in AI outputs [Maynez et al., 2020; Ji et al., 2023] focuses on factual grounding in generated text. source-grounding-audit extends this to harness content specifically, where phantom claims are particularly consequential because harness documentation is treated as ground truth by the orchestrating AI. Ning et al. [arXiv:2605.25665] introduce a two-pass meta-detector pattern that validates initial detection results through a second independent check, reducing false positives in harness validation workflows — a pattern steel-quench adopts in its Wave structure.

2.3 Self-Evolving Systems

Reflexion [Shinn et al., 2023] and Self-Refine [Madaan et al., 2023] enable agents to revise their own outputs through verbal feedback. harvest-loop applies analogous self-refinement at the harness level — not to individual agent outputs but to the persistent skill infrastructure across sessions. The key distinction is that harvest-loop targets cross-session knowledge accumulation rather than within-session output revision. Liu et al. [arXiv:2605.26302] identify compression aging as a primary failure mode in cross-session knowledge systems — when context compresses, earlier learnings vanish from LLM memory. harvest-loop Step 0-a addresses this by requiring real-time completion logging during the session, preventing completed work from incorrectly reappearing in next-session priority lists.

2.4 Positioning vs. Automated Harness Optimization

harness-evolver [Christi, 2026] implements the Meta-Harness pattern in a 7-stage automated loop requiring LangSmith instrumentation and Python infrastructure, targeting automated optimization of harness code at benchmark scale. forge-harness targets a complementary layer: harness knowledge validation and evolution for practitioners who prioritize human-approved architectural gates and zero additional infrastructure. These are not competing approaches — automated code optimization (harness-evolver) and principled knowledge engineering (forge-harness) address different layers of the same problem.

2.5 Empirical Validation: Multi-Model Review Convergence

A central claim in harness engineering is that LLM outputs converge given similar inputs, and differentiation comes from the wrapper infrastructure — the harness layer — rather than from model selection alone. We tested this claim empirically through a controlled comparison of three frontier models (Opus 4.7, GPT-5.5, Sonnet 4.6) reviewing the same harness artifact.

Experimental Setup: We provided identical input — a hallucinate library skeleton (14 files) and four arXiv papers on harness meta-engineering — to three models in complete context isolation. Group A used gh copilot sidecar mode (external process) to invoke Opus 4.7 and GPT-5.5 independently. Group B used the Agent tool dispatch pattern within Claude Code to invoke Sonnet 4.6 three times independently. All models operated without access to the primary session context.

Results: The four S-grade improvement recommendations showed 100% consensus across all three models: (1) four-way Verdict enum (TP/FP/UNKNOWN/CONFLICT), (2) MetaDetector two-pass validation hook, (3) README/API misalignment removal, and (4) LICENSE + test suite inclusion. The models differed only in phrasing and secondary details (~10% variance). Context isolation level (external process vs. Agent subcontext) had no measurable effect on priority identification.

Implication: Given consistent input quality — in this case, complete skeleton code and targeted literature review — frontier LLMs converge on the same high-priority recommendations. The harness layer, not the model, determines output quality. This finding validates a core design principle: harness engineering efforts compound across model upgrades. When Opus 5 or GPT-6 releases, the accumulated harness patterns remain valid; only the underlying model swaps out. Model selection is interchangeable; prompt design and harness structure are not.

Environment-Dependent Value: This convergence finding applies when models are directly accessible. In corporate environments where Claude Code operates via Bedrock API with Sonnet-only access, gh copilot sidecar remains the sole path to Opus 4.7 reasoning for architectural decisions. In such constrained environments, sidecar mode remains S-grade; in unconstrained environments, it is optional for token distribution.

---

3. Methods

3.1 steel-quench — Adversarial Validation (F1)

steel-quench addresses F1 (self-referential validation) through strict structural separation of attack and defense across convergence rounds.

Wave 1 — Devil Attack: An adversarial agent (devil) operates in isolation with access only to static harness files. It applies five mandatory attack angles: (1) existence justification — "why this structure?"; (2) documentation-code alignment — "do documented and actual behaviors match?"; (3) bus factor — "can operations continue without the primary maintainer?"; (4) platform obsolescence — "does the architecture survive ecosystem changes?"; (5) self-referential structure — "is there a closed circuit where the system evaluates itself?" Each finding is classified S (immediate blocker), A (required before deployment), or B (improvement recommended). Abstract critique without file-level citation is rejected.

Wave 2 — Defense: Defense draws on three principles: (1) *external evidence precedence* — living system history (deployment logs, external PRs, user adoption) that the isolated devil cannot observe; (2) *implementation priority* — a concrete fix is stronger evidence than a defensive argument; (3) *explicit residual risk* — unresolved defects are named and carried forward, not silently dropped.

The Sandboxed Adversary Principle: The fundamental asymmetry of steel-quench is that the devil operates in an information sandbox. It sees static artifacts; the defense can cite dynamic evidence — adoption rates, external contributor PRs, multi-project deployment history. As Waves deepen, the devil exhausts its structurally accessible attack surface. This is why the convergence criterion (zero new S-grade) is theoretically grounded: it marks the point at which the adversary's accessible attack space is exhausted, not merely a threshold chosen empirically.

Wave 3+ — Convergence: Convergence is declared when no new S-grade blockers appear. A/B-grade residual items are captured as named technical debt.

Wave 4 — Meta-Aware Adversary: An optional depth extension for high-stakes validation. The Wave 4 devil is explicitly told it is an AI with hallucination risk, context window limits, and tool dependencies — and is instructed to exploit these characteristics as attack vectors against five AI-specific angles: API single point of failure, context collapse, prompt injection exposure, hallucination-contaminated defense claims, and tool dependency lock-in.

Wave 5 — Post-Fix Verification (Regression Guard): After all S/A-grade defects are remediated, a final verification pass confirms no regressions were introduced during the fix cycle. Regression guard applies six checks: (1) frontmatter consistency — required fields present and well-formed; (2) critical section presence — core functionality sections not deleted; (3) code block syntax — no broken fence markers; (4) keyword stability — primary skill triggers and identifiers preserved; (5) reference integrity — internal links remain valid; (6) line count direction — file shrinks only when decorative content is removed, not core functionality. Findings are graded M (mandatory block), S (serious warning), R (recommended review). M > 0 triggers immediate rollback; S > 0 requires human review; 0 = deployment approved.

3.2 source-grounding-audit — Phantom Detection (F2)

source-grounding-audit addresses F2 (phantom claim accumulation) through mandatory evidence citation for factual claims in harness content.

Phantom Claim Taxonomy: We identify three categories of phantom claims in AI-generated harness content:

- **File phantoms**: A referenced file, function, or configuration value does not exist at the cited path. Generated harness documentation frequently asserts "see `path/to/file.md` for details" when no such file exists.
- **Capability phantoms**: A documented skill or agent behavior has never been implemented or verified in a cold-start execution. The SKILL.md describes behavior that was designed but never exercised.
- **Measurement phantoms**: A cited metric, benchmark result, or performance figure has no corresponding measurement artifact — no log file, no test output, no commit containing the measurement. The figure is LLM-reconstructed plausibility.

Detection Method: For each factual claim in harness content, source-grounding-audit applies a two-step check: (1) *existence check* — does the referenced artifact exist at the cited location? (2) *origin check* — is the claim derivable from a non-LLM source (file contents, commit history, external measurement)? Claims that fail either check are classified as phantom candidates and flagged for remediation or removal.

The Defense Evidence Standard: source-grounding-audit establishes an evidence standard for steel-quench Wave 2 specifically: defense claims must cite an original file path, a commit hash, or a measured value. "LLM-assessed" or "model-inferred" is explicitly not accepted as defense evidence. This prevents the hallucination contamination pattern (P7) in which Wave 2 defense arguments inherit the errors of the harness content they are defending.

Applied Protocol:

```
For each SKILL.md claim of form "X achieves Y" or "Z exists at path P":
  1. Verify Z at P (file existence check)
  2. Verify X→Y causal link has a measurement artifact
  3. Flag: GROUNDED (artifact exists) / PHANTOM (no artifact) / UNVERIFIABLE (external)
  Phantom-rate threshold for deployment: < 10% of claims
```

3.3 harvest-loop — Self-Evolution Pipeline (F3)

harvest-loop addresses F3 (session learning loss) through a structured pipeline that transforms session artifacts into validated harness skill upgrades.

Pipeline Architecture:

```
 Session end
    │
    ▼
[Step 0-a] Compression Loss Defense
    │  Before harvesting: append completed S/A-grade items to session log
    │  → Prevent LLM memory loss when context compresses
    │
    ▼
[Step 0-b] Cross-Check Gate
    │  Does this session's "next-session starter card" still list items
    │  that were actually completed this session?
    │  → Remove completed items before new priorities overwrite them
    │
    ▼
[Step 1] field-harvest
    │  Scan session git diff + outputs
    │  → Extract patterns as absorption candidates (3+ required to proceed)
    │
    ▼
[Step 2] contention-layer
    │  Each candidate pattern ↔ existing skills
    │  → Overlap ≥ 70%: existing skill enhancement candidate
    │  → Gap: new skill candidate
    │  → Two existing skills conflict: mediation skill candidate
    │
    ▼ (parallel)
[Step 3a] devil-advocate         [Step 3b] innovator
    │  Attack existing skills          │  Propose new skills / enhancements
    │  using session findings          │  with Done-When criteria (required)
    │
    └────────────────┬─────────────────┘
                     ▼
[Step 3.5] synthesizer gate  ← key innovation
    │  Cross-validate devil × innovator results:
    │  · S-grade attack + matching proposal → HIGH (attack validates proposal)
    │  · S-grade attack + no proposal → HIGH (existing skill has unaddressed weakness)
    │  · No attack + proposal only → MED (unvalidated; defer to next session)
    │  · Attack invalidates proposal premise → REJECT (proposal dropped)
    │
    ▼
[Step 4] harness-doctor
    │  Check: Done-When exists? Overlap < 70%? No self-reference?
    │
    ▼
[Step 5] verify-bidirectional
    │  Bidirectional link check: if A references B, does B reference A?
    │
    ▼
[Step 10] Post-Fix Regression Guard
    │  After implementing approved upgrades: verify no regressions
    │  → 6-axis check (frontmatter, sections, code, keywords, refs, line count)
    │  → M/S/R-tier judgment: M>0 = BLOCK / S>0 = REVIEW / 0 = PASS
    │
    ▼
User approval gate → implement or archive as signal
```

The Synthesizer Gate: The critical innovation in harvest-loop is Step 3.5, which cross-validates devil-advocate attack results against innovator proposals before any skill is created or modified. Without this gate, devil and innovator operate in parallel but never converge — attacks may invalidate the premises of proposed enhancements without the proposer being aware. The synthesizer gate forces explicit resolution of this conflict, producing a ranked candidate list with structural justification for each grade assignment.

Contention as Signal: contention-layer treats skill conflicts not as problems to eliminate but as generative signals. When two existing skills conflict on handling the same scenario, this conflict marks an underspecified boundary — a candidate location for a new mediating skill. This converts the most common source of harness technical debt (overlapping skill coverage) into a harness growth mechanism.

Compression Aging Defense: Step 0-a addresses a specific failure mode in multi-session harness evolution: when LLM context compresses, completed work from the current session may vanish from memory. The next session's "starter card" (a next-priority checklist) can incorrectly retain items that were finished but never marked complete due to compression loss. Step 0-a requires real-time completion logging during the session — not retrospective LLM recall. Step 0-b cross-checks this log against the starter card before overwriting priorities, preventing completed items from reappearing as next-session blockers.

3.4 sim-conductor — Pre-Deployment Simulation (F4)

sim-conductor addresses F4 (unvalidated transfer) through structured simulation of non-owner harness adoption before cascade deployment.

Area A — External User Simulation: A cold-start external user persona is simulated: no author context, no shared vocabulary history, no access to session transcripts. The simulation attempts to: (1) trigger skills using natural vocabulary (not the author's trigger phrases); (2) reach expected outcomes via the documented workflow; (3) identify dead ends where harness vocabulary creates barriers for unfamiliar users.

Area B — Internal Consistency Simulation: A simulation of consistent harness operation across a multi-session arc, verifying that memory files, skill trigger conditions, and agent dispatch rules remain self-consistent over time without human correction.

Cascade Gate: sim-conductor Area A must achieve ≥ 80% skill reachability via natural vocabulary before cascade deployment is approved. This prevents the common failure mode in which a harness works for its author and fails silently for the first adopter.

3.5 Future Work: Harness Federation Sync (Preview)

A remaining gap in harness engineering is cross-node synchronization: when multiple independent harness deployments (a public reference implementation, an internal enterprise fork, a meta-development node) evolve independently, improvements discovered in one node do not automatically propagate to others. Manual sync cycles require human diagnosis → prescription → verification → federation, consuming 15–20 minutes per cycle.

harness-federation-sync addresses this gap through automated bidirectional sync across three independent nodes: forge-harness (external public), pay-meta-harness (enterprise deployment), and claude-chrono (meta-development node). Each node maintains sovereignty — independent commit authority, divergent evolution paths — while automatically syncing S/A-grade improvements through steel-quench + phantom detection pipelines. Expected reduction: manual 17-minute cycle → automated 5-minute cycle. Implementation and validation remain future work; the design is presented here as a preview of the next methodological layer.

---

4. Evaluation

We applied all four methods to **forge-harness** [chrono-code, 2026], a meta-harness toolkit for Claude Code comprising 23 skills, 3 agents, and 2 plugin namespaces (fh-meta, fh-commons). Applying forge-harness methods to forge-harness itself constitutes a self-verification scenario (failure mode F1). We address this through external validation (Section 4.3).

4.1 steel-quench Results: Wave 1–3

The devil agent identified 10 findings:

Attack Angle	Grade	Finding
Documentation-code alignment	S	CI agent count used `find -type d`; agents are `.md` files → CI always reported 0 agents
Documentation-code alignment	S	README referenced `~/PycharmProjects/` (internal path) in 8 locations
Self-referential structure	S	`plugin.json` version pinned to `0.0.0` with no CI enforcement
Self-referential structure	S	README self-marketing claims without external citation
Bus factor	A	All skills authored by single contributor; no cold-start validation
Self-referential structure	A	Author placeholder `your-name@example.com` in published config
Self-referential structure	A	No `engines.claudeCode` compatibility field
Existence justification	A	29 skill activations with no explicit benefit-scaling justification
Platform obsolescence	B	No degradation path if Claude Code plugin API changes
Self-referential structure	B	Internal-only plugin referenced without external equivalent

All 4 S-grade blockers were immediately remediated. All 4 A-grade items resolved within the same session. Wave 3: zero new S-grade blockers. **Convergence achieved.**

4.2 source-grounding-audit Results

Applying source-grounding-audit to forge-harness SKILL.md content identified 3 phantom capability claims:

- **Capability phantom**: `sim-conductor Area B` documented as fully operational; cold-start execution confirmed the multi-session arc simulation had never been exercised end-to-end.
- **Measurement phantom**: README claimed a specific token reduction figure without a corresponding measurement artifact. Claim removed; replaced with architectural description only.
- **Capability phantom**: `verify-bidirectional` described link-checking as automatic; inspection confirmed the check requires manual invocation per link pair.

Phantom rate before audit: 3/47 documented claims (6.4%). After remediation: 0/44 (0%). Three claims removed rather than fabricating evidence.

4.3 harvest-loop Results

Five external pull requests (#1–#5) submitted during the evaluation session were processed through harvest-loop. The synthesizer gate produced the following grade assignments:

PR	Pattern	Synthesizer Grade	Outcome
#1	CI agent counting fix + path neutralization	HIGH	Immediate integration
#2	arXiv study ingestion → frontier digest absorption	HIGH	README external validation block
#3	Internal placeholder removal	MED	Applied; no skill upgrade generated
#4	harness-evolver cross-audit → sister asset protocol	HIGH	SKILL.md positioning section added
#5	Regression guard + worktree isolation + numeric scoring	HIGH	Three SKILL.md upgrades (v1.2)

HIGH-grade patterns (4/5) resulted in immediate skill upgrades. The one MED-grade pattern (PR #3) produced a harness change without generating a reusable skill candidate, consistent with the synthesizer gate's "no devil validation → MED" classification.

4.4 sim-conductor Results

sim-conductor Area A was applied before cascade deployment (Section 4.5). The simulation identified two vocabulary barriers: (1) the skill trigger `/steel-quench` was reachable by the author's vocabulary but not by the natural phrasing "adversarial review" or "devil's advocate check" used by simulated external users; (2) `harvest-loop` was not triggered by "session cleanup" or "wrap up" phrasing common among non-author practitioners.

Both barriers were resolved by expanding trigger phrase lists before cascade deployment. Post-fix simulation: 26/26 skills (100%) reachable via natural vocabulary across 4 simulated external user profiles.

4.5 External Validation and Cascade Deployment

To address the self-referential validation concern (F1), external validation was conducted through an independent GitHub account operating on a separate network with a fresh repository clone and no shared session context. This account submitted five PRs independently, with one additional defect identified (CI agent counting bug) that primary-environment validation had missed — confirming that external perspective surfaces blind spots not accessible from within the primary environment.

Beyond the evaluation environment, forge-harness was autonomously adopted by three additional practitioners across different projects. One practitioner applied steel-quench independently to a QA automation tool, identified two structural defects, and submitted corrections without author assistance. This non-owner autonomous operation — cascade deployment — constitutes observational validation that all four methods transfer without the originating author's presence.

4.6 Independent Architectural Convergence

Frontier search during the evaluation identified harness-evolver [Christi, 2026] — an independent Claude Code plugin implementing a 7-stage harness optimization loop with 6 agents. Cross-audit confirmed that harness-evolver and forge-harness independently converged on the same outer-loop architecture: field observation → adversarial critique → synthesis → integration → verification. The Stanford IRIS Lab Meta-Harness [arXiv:2603.28052] implements the same outer-loop principle at automated benchmark scale. Three independent implementations arriving at the same loop structure through different paths validates the architecture as a structural property of the harness engineering problem rather than an artifact of any single implementation's assumptions.

---

5. Discussion

5.1 Limitations

Single-project evaluation: All four methods are evaluated on forge-harness itself. While the self-verification scenario is methodologically challenging (addressed through external validation), results from a single project cannot establish generalizability. Multi-project controlled evaluation is required.

External validation independence: The "external" account used in Section 4.3 is operated by the same author as the primary account, from a different network and session context but not an independent practitioner. Truly independent external validation would require practitioners with no prior knowledge of forge-harness.

Phantom audit coverage: source-grounding-audit was applied to a subset of SKILL.md documentation. Full coverage of all 23 skills and 3 agents was not completed; the 6.4% phantom rate should be understood as a partial-coverage estimate.

Cascade deployment: Section 4.5 reports observational adoption. Effect sizes, error rates relative to baselines, and longitudinal stability have not been measured.

Wave 4 coverage: Meta-aware adversary mode (Section 3.1) is documented in the framework and applied in practitioner settings but Wave 4 results are not reported in this evaluation.

Human-in-the-loop ceiling: forge-harness intentionally preserves human approval gates at each pipeline stage. This means forge-harness cannot achieve the fully automated optimization performance demonstrated by harness-evolver. The human-in-the-loop design is a deliberate architectural choice — not a limitation — but practitioners requiring fully automated harness code search at benchmark scale should evaluate harness-evolver instead.

5.2 When to Apply Each Method

Situation	Recommended Method
Pre-deployment validation of a complex harness	steel-quench Wave 1–3
Harness content has grown through AI-generated documentation	source-grounding-audit
Working sessions generate repeated learnings that are not persisted	harvest-loop
First non-owner adoption is imminent	sim-conductor Area A
Post-Wave-3 high-stakes release (public listing, academic submission)	steel-quench Wave 4

The four methods are complementary and designed to run in the order listed: structural defects (steel-quench) → phantom claims (source-grounding-audit) → knowledge evolution (harvest-loop) → transfer validation (sim-conductor).

5.3 The Harness Engineering Layer

The four methods in this paper operate at a layer distinct from both prompt engineering (individual instruction quality) and model evaluation (benchmark performance). They address the structural properties of the persistent environment around the AI — the rules, memory, and skills that persist across sessions and shape every interaction. As AI harnesses become the primary determinant of collaboration quality, engineering methods for this layer become correspondingly important. forge-harness is one contribution toward a more principled harness engineering practice.

---

6. Conclusion

We presented forge-harness, a toolkit of four methods addressing the primary failure modes in AI collaboration harness engineering: steel-quench for adversarial structural validation, source-grounding-audit for phantom claim detection, harvest-loop for session-to-harness self-evolution, and sim-conductor for pre-deployment transfer validation. Applied to forge-harness itself, the methods resolved 10 structural defects, detected and removed 3 phantom claims, absorbed 5 external contributions as validated upgrades, and confirmed multi-user autonomous deployment — all within a single evaluation session. Independent architectural convergence across three implementations (forge-harness, harness-evolver, Stanford Meta-Harness) validates the outer-loop structure as a robust solution pattern. Multi-model convergence testing (Opus 4.7, GPT-5.5, Sonnet 4.6) confirms that given consistent input quality, frontier LLMs converge on the same high-priority recommendations — validating the harness-layer differentiation thesis and the long-term value of harness pattern accumulation across model upgrades.

The methods share a common design principle: each separates the roles of generator and evaluator that typically collapse in solo harness development. steel-quench separates attack from defense. source-grounding-audit separates claim from evidence. harvest-loop separates observation from absorption through the synthesizer gate. sim-conductor separates author perspective from adopter perspective. This separation is the structural mechanism by which the methods surface what informal self-evaluation cannot.

Artifact availability: https://github.com/chrono-code/forge-harness

---

References

[arXiv:2604.14228] Chen, X. et al. "98.4% of Claude Code is Harness Infrastructure." VILA-Lab, 2026.

[arXiv:2605.18747] [43 authors]. "Code as Agent Harness." 2026.

[arXiv:2604.21003] Sylph.AI. "The Last Harness You'll Ever Build." 2026.

[arXiv:2603.28052] Stanford IRIS Lab. "Meta-Harness: Automated Harness Optimization via Outer-Loop Execution." 2026.

[arXiv:2605.25665] Ning, X. et al. "Meta-Engineering Frameworks for AI Agent Harnesses: Two-Pass Validation and Four-Way Arbitration." UIUC, 2026.

[arXiv:2605.26302] Liu, S. et al. "AgingBench: Measuring Harness Knowledge Degradation Across Sessions." 2026.

[arXiv:2605.27575] Min, K. et al. "Agyn: Signal-Driven Harness Adaptation for Infrastructure-as-Code." 2026.

[arXiv:2605.28065] Park, J. et al. "SpatialBench: Joint Evaluation of Model-Harness Pairs in Agent Performance." 2026.

[Christi, 2026] Christi, R. harness-evolver. MIT License. https://github.com/raphaelchristi/harness-evolver, 2026.

[Perez et al., 2022] Perez, E. et al. "Red Teaming Language Models with Language Models." arXiv:2202.03286.

[Bai et al., 2022] Bai, Y. et al. "Constitutional AI: Harmlessness from AI Feedback." arXiv:2212.08073.

[Maynez et al., 2020] Maynez, J. et al. "On Faithfulness and Factuality in Abstractive Summarization." ACL 2020.

[Ji et al., 2023] Ji, Z. et al. "Survey of Hallucination in Natural Language Generation." ACM Computing Surveys, 2023.

[Shinn et al., 2023] Shinn, N. et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." NeurIPS 2023.

[Madaan et al., 2023] Madaan, A. et al. "Self-Refine: Iterative Refinement with Self-Feedback." NeurIPS 2023.

Changelog

v0.6 (2026-05-28): - **Added**: Section 2.5 — Empirical Validation: Multi-Model Review Convergence (LLM Core convergence thesis with 100% S-grade consensus across Opus/GPT/Sonnet) - **Added**: Section 3.1 Wave 5 — Post-Fix Verification (Regression Guard with 6-axis M/S/R-tier judgment) - **Added**: harvest-loop Steps 0-a/0-b — Compression Loss Defense and Cross-Check Gate - **Added**: harvest-loop Step 10 — Post-Fix Regression Guard - **Added**: Section 3.5 — Future Work: Harness Federation Sync (preview) - **Improved**: Section 2.1 — Added four new arXiv references (2605.25665, 2605.26302, 2605.27575, 2605.28065) - **Improved**: Section 2.2 — Integrated Ning et al. two-pass meta-detector pattern - **Improved**: Section 2.3 — Integrated Liu et al. compression aging mechanism - **Improved**: Section 6 — Updated conclusion with multi-model convergence validation

Stats (v0.5, 2026-05-26 ~ 2026-05-28): - Zenodo: 49 views / 33 downloads (29 unique) in 2 days, no promotion

---

Draft v0.6 — 2026-05-28.